TỐI ƯU HÓA MÃ HÓA SAT CHO BÀI TOÁN TÔ MÀU ĐỒ THỊ (GCP) VÀ ĐÁNH GIÁ SO SÁNH HIỆU NĂNG VỚI CÁC BỘ GIẢI CÔNG NGHIỆP

Nguyễn Đức Mạnh, Hà Phú Duy, Đặng Nhật Quang

Khoa Công nghệ Thông tin, Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội
144 Xuân Thủy, Cầu Giấy, Hà Nội, Việt Nam

THÔNG TIN BÁO CÁO

Từ khóa:
Graph Coloring Problem (GCP), Chromatic Number, Zykov's Tree, CEGAR, Incremental SAT, PySAT, At-Most-One Encoding, ILP, CP, CDCL.

TÓM TẮT

Bài toán Tô màu Đồ thị (Graph Coloring Problem – GCP) là bài toán tối ưu hóa tổ hợp và thỏa mãn ràng buộc (CSP) kinh điển thuộc lớp NP-khó trong Khoa học Máy tính. Báo cáo này trình bày giải pháp giải chính xác bài toán tìm sắc số $\chi(G)$ của đồ thị bằng cách chuyển đổi sang bài toán Thỏa mãn Công thức Logic Mệnh đề (SAT) thông qua thư viện `PySAT`. Chúng tôi phân tích và đánh giá thực nghiệm 5 kỹ thuật mã hóa At-Most-One (AMO) cho mô hình trực tiếp (Pairwise/Binomial, Sequential Counter, Binary, Commander, Product Encoding) kết hợp với phương pháp mã hóa ghép cặp đỉnh dựa trên cây Zykov và khung giải lặp tăng trưởng CEGAR (Incremental SAT) từ bài báo gốc. Đồng thời, báo cáo tiến hành thử nghiệm so sánh trực diện hiệu năng giữa bộ giải SAT (CaDiCaL, Glucose) với các công cụ giải công nghiệp hàng đầu thuộc trường phái Lập trình Tuyến tính Nguyên (ILP: Gurobi, CPLEX MIP) và Lập trình Ràng buộc (CP: CPLEX CP, Google OR-Tools CP-SAT) trên bộ dữ liệu chuẩn DIMACS/COLOR03. Kết quả cho thấy việc kết hợp kỹ thuật mã hóa tối ưu và cơ chế CEGAR Incremental SAT giúp triệt tiêu hiện tượng bùng nổ mệnh đề bậc ba $O(n^3)$, vượt qua các hệ giải ILP/CP thương mại trên các đồ thị quy mô lớn.

1 Mở đầu (Introduction)

1.1 Đặt bài toán và Động lực nghiên cứu

Bài toán Tô màu Đồ thị (Graph Coloring Problem – GCP) yêu cầu gán một số lượng màu tối thiểu cho tất cả các đỉnh của một đồ thị vô hướng $G = (V, E)$ (với $|V| = n$ đỉnh và tập cạnh $E \subseteq V \times V$) sao cho không có hai đỉnh kề nhau nào (hai đỉnh được nối với nhau bởi một cạnh $(u, v) \in E$) nhận cùng một màu [1, 8]. Số lượng màu nhỏ nhất cần thiết để tô màu hợp lệ cho đồ thị G được gọi là sắc số (chromatic number) của đồ thị và ký hiệu là $\chi(G)$ [1].

Bài toán GCP không chỉ mang ý nghĩa lý thuyết đồ thị sâu sắc mà còn có ứng dụng thực tiễn quan trọng trong nhiều lĩnh vực kỹ thuật:

- Lập lịch tác vụ và phân bổ thanh ghi trong tối ưu hóa trình biên dịch (Register Allocation in Compiler Optimization) [9, 10].
- Xếp lịch thi và lập thời khóa biểu tự động (Exam Timetabling, Sports Scheduling) [11, 8].
- Giải các bài toán đố logic tổ hợp (như Sudoku) và phân chia tần số mạng viễn thông [1, 8].

1.2 Nhược điểm của các phương pháp truyền thống

Việc xác định sắc số $\chi(G)$ của một đồ thị là một bài toán NP-khó (NP-hard), thậm chí NP-khó để xấp xỉ trong nhiều kịch bản tổng quát [1]. Các phương pháp giải bài toán GCP thường được chia thành hai nhóm:

- **Các phương pháp không đầy đủ (Incomplete methods):** Dựa trên thuật toán tham lam (như DSatur [5]) hoặc meta-heuristic, có khả năng xử lý đồ thị lớn nhưng chỉ tìm được cận trên (upper bound) và không đảm bảo chứng nhận được tính tối ưu của nghiệm [1].
- **Các phương pháp đầy đủ truyền thống (Complete methods):** Dựa trên mô hình Nhánh và Cận (Branch and Bound) hoặc Lập trình Ràng buộc (CP) gán miền giá trị màu cho từng đỉnh [4, 2]. Do các màu có tính hoán vị tương đương nhau (interchangeability of colours), cách biểu diễn này sinh ra số lượng lớn cấu hình đối xứng ($k!$ hoán vị màu), làm bùng nổ không gian tìm kiếm khi số đỉnh n và số màu k tăng cao [1, 3].

1.3 Phương pháp tiếp cận SAT Solving

Việc chuyển đổi bài toán k-Colouring sang dạng Chuẩn Hội (CNF) và giải bằng các bộ giải SAT hiện đại dựa trên thuật toán CDCL (Conflict-Driven Clause Learning) mở ra hướng tiếp cận hiệu quả để chứng nhận nghiệm tối ưu [12, 6, 7]. Tuy nhiên, mã hóa trực tiếp cơ sở (Basic Encoding) vẫn chịu ảnh hưởng bởi tính đối xứng màu và bùng nổ mệnh đề khi sử dụng mã hóa Pairwise [13], trong khi mã hóa dựa trên cây Zykov (ghép cặp các đỉnh không kề nhau có cùng màu) tuy phá vỡ được tính đối xứng nhưng lại đòi hỏi tới $O(n^3)$ mệnh đề bậc ba để đảm bảo tính bắc cầu (transitivity) và tính Euclid (Euclideanity) [1, 3, 2]. Để khắc phục nhược điểm này, phương pháp tiếp cận Incremental SAT dựa trên khung CEGAR (Counter-Example Guided Abstraction Refinement) [16, 1] chỉ nạp một phần ràng buộc (under-abstraction) và bổ sung tăng trưởng các mệnh đề bị vi phạm theo từng vòng lặp.

Đóng góp chính của báo cáo bao gồm:

1. Biểu diễn và cài đặt hoàn chỉnh mô hình SAT CNF trực tiếp cho bài toán GCP (sử dụng 5 kỹ thuật mã hóa AMO: Pairwise/Binomial, Sequential Counter, Binary, Commander, và Product Encoding) dựa trên thư viện `PySAT` [17, 18].
2. Cài đặt và đánh giá phương pháp mã hóa dựa trên cây Zykov theo hai chế độ: gọi SAT nhiều lần (Full Encoding với tìm kiếm tuyến tính `1toN`, `Nto1`, nhị phân `Dicho`) và Incremental SAT kết hợp tinh chỉnh CEGAR [1].

3. Đánh giá hiệu năng đối đầu trực tiếp giữa các biến thể SAT Solvers [19, 20] với các công cụ giải chính xác công nghiệp gồm ILP (Gurobi [21], CPLEX MIP [22]) và CP (CPLEX CP [22], Google OR-Tools CP-SAT) trên bộ dữ liệu benchmark chuẩn.

2 Mô hình hóa Toán học (Mathematical Formulations)

2.1 Biểu diễn Biến nhị phân (Boolean Variables)

Cho đồ thị vô hướng $G = (V, E)$ với tập đỉnh $V = \{1, 2, \ldots, n\}$ và số màu thử nghiệm k. Báo cáo khảo sát hai cách biểu diễn biến mệnh đề nhị phân:

1. Biểu diễn trực tiếp theo đỉnh – màu (Direct Vertex-Colour Encoding): Gọi $x_{v,i} \in \{0, 1\}$ là biến mệnh đề nhị phân đại diện cho việc gán màu $i \in \{1, \ldots, k\}$ cho đỉnh $v \in V$ [1]:

$$x_{v,i} = \begin{cases} 1, & \text{nếu đỉnh } v \text{ được tô màu } i \\ 0, & \text{ngược lại} \end{cases} \tag{1}$$

2. Biểu diễn ghép cặp đỉnh theo cây Zykov (Zykov Pairing Encoding): Dựa trên hệ thức đệ quy co đỉnh – thêm cạnh của Zykov [1, 3]:

$$\chi(G) = \min\{\chi(G/uv), \chi(G + uv)\} \quad \forall (u, v) \notin E \tag{2}$$

Thay vì gán màu trực tiếp, ta sử dụng tập biến ghép cặp $S = \{s_{u,v} \mid u, v \in V, u < v\}$ và tập biến đếm màu mới $C = \{c_2, c_3, \ldots, c_n\}$ [1]:

$$s_{u,v} = \begin{cases} 1, & \text{nếu hai đỉnh } u \text{ và } v \text{ có cùng màu (co đỉnh } G/uv) \\ 0, & \text{nếu hai đỉnh } u \text{ và } v \text{ khác màu (thêm cạnh } G + uv) \end{cases} \tag{3}$$

$$c_j = \begin{cases} 1, & \text{nếu đỉnh } j \text{ cần một màu mới chưa xuất hiện ở các đỉnh } i < j \\ 0, & \text{ngược lại} \end{cases} \tag{4}$$

2.2 Mã hóa SAT CNF (Exact-One Constraint)

Trong mô hình mã hóa trực tiếp, mỗi đỉnh $v \in V$ phải nhận đúng một màu (Exact-One – EXO), được tách thành hai điều kiện sơ cấp: At-Least-One (ALO) và At-Most-One (AMO), kết hợp cùng ràng buộc cạnh kề và mô hình Zykov/CEGAR. Sơ đồ luồng xử lý bài toán được thể hiện trong Hình 1.



Hình 1: Sơ đồ luồng kiến trúc giải Bài toán Tô màu Đồ thị (GCP) bằng phương pháp SAT Solving.

2.2.1 Ràng buộc At-Least-One (ALO)

Trong mô hình trực tiếp, mỗi đỉnh $v \in V$ phải được tô ít nhất một màu trong k màu, đồng thời hai đỉnh kề nhau $(u, v) \in E$ không được cùng màu [1]:

$$\bigwedge_{v \in V} \left(\bigvee_{i=1}^{k} x_{v,i} \right) \quad \text{(Mỗi đỉnh có ít nhất 1 màu – ALO)} \tag{5}$$

$$\bigwedge_{(u,v) \in E} \bigwedge_{i=1}^{k} (\neg x_{u,i} \vee \neg x_{v,i}) \quad \text{(Hai đỉnh kề nhau khác màu)} \tag{6}$$

Đối với mô hình cây Zykov trong bài báo tham khảo [1], công thức đầy đủ $tr(G)$ bao gồm các mệnh đề đơn cấm ghép cặp hai đỉnh kề nhau và các mệnh đề bậc ba đảm bảo tính nhất quán đường đi (path consistency):

$$\neg s_{u,v} \quad \forall (u, v) \in E, u < v \tag{7}$$

$$\text{transitivity}(i, j, k) := (\neg s_{i,j} \vee \neg s_{j,k} \vee s_{i,k}) \quad \forall i, j, k \in V, i < j < k \tag{8}$$

$$\text{euclideanity}(i, j, k) := (\neg s_{i,j} \vee \neg s_{i,k} \vee s_{j,k}) \quad \forall i, j, k \in V, i < j < k \tag{9}$$

Số màu sử dụng bởi phép ghép cặp hợp lệ là $\Psi(I_S) = 1 + \sum_{j=2}^{n} c_j$, trong đó biến c_j được định nghĩa qua cổng AND [1]:

$$c_j \Leftrightarrow \bigwedge_{s_{i,j} \in S, i<j} \neg s_{i,j} \quad \forall 1 < j \leq n \tag{10}$$

Để quyết định xem $\chi(G) \leq k$, ta bổ sung ràng buộc lực lượng $\sum_{j=2}^{n} c_j \leq k - 1$ bằng mạng lực lượng (Cardinality Network) [15]. Trong phiên bản **Incremental SAT (CEGAR)**, ta lược bỏ các mệnh đề transitivity và euclideanity khỏi công thức ban đầu để tạo công thức xấp xỉ dưới $\tilde{\phi}$, sau đó dùng bộ kiểm tra đa thức (`check`) duyệt từng đỉnh để phát hiện các bộ ba (u, v, w) vi phạm và thêm tăng trưởng vào bộ giải SAT [1].

2.2.2 Phân tích 5 Kỹ thuật Mã hóa At-Most-One (AMO)

Để đảm bảo mỗi đỉnh $v \in V$ nhận tối đa một màu trên tập biến $X_v = \{x_{v,1}, x_{v,2}, \ldots, x_{v,k}\}$ (với $m = k$), ta khảo sát 5 kỹ thuật mã hóa AMO [13, 14]:

1. **Pairwise / Binomial Encoding:**

$$\bigwedge_{1 \leq i < j \leq m} (\neg x_{v,i} \vee \neg x_{v,j}) \tag{11}$$

   Không sử dụng biến phụ trợ (0 biến phụ trợ), số lượng mệnh đề sinh ra cho mỗi đỉnh là $\binom{m}{2} = O(m^2)$.

2. **Sequential Counter Encoding (Sinz, 2005) [14]:** Xây dựng mạch đếm lũy tiến sử dụng các biến phụ trợ s_i ($s_i = 1$ nếu có ít nhất 1 biến $x_{v,r} = 1$ với $r \leq i$). Cần $m - 1$ biến phụ trợ ($O(m)$ biến phụ), độ phức tạp số mệnh đề giảm xuống $O(m)$.

3. **Binary (Bitwise) Encoding:** Mã hóa chỉ số màu của mỗi đỉnh thành chuỗi bit nhị phân độ dài $\lceil \log_2 m \rceil$. Cần $O(\log_2 m)$ biến phụ trợ và sinh ra $O(m \log_2 m)$ mệnh đề cho mỗi đỉnh.

4. **Commander Encoding:** Chia m biến màu của đỉnh thành các nhóm nhỏ kích thước g, mỗi nhóm gán một biến chỉ huy (commander variable) để quản lý trạng thái kích hoạt. Cần $O(m/g)$ biến phụ trợ và $O(m)$ mệnh đề.

5. **Product Encoding (Greshuk et al.):** Chiếu chỉ số màu $r \in \{1, \dots, m\}$ lên lưới ma trận 2 chiều kích thước $p \times q$ với $p = \lceil \sqrt{m} \rceil$ và $q = \lceil m/p \rceil$, tương ứng với tọa độ (a, b). Tạo p biến phụ trợ hàng r_a và q biến phụ trợ cột c_b (tổng cộng $O(\sqrt{m})$ biến phụ trợ). Mệnh đề liên kết biến gốc $x_{v,r}$ với biến phụ trợ được thiết lập qua quan hệ kéo theo $x_{v,r} \to (r_a \wedge c_b)$:
$$(\neg x_{v,r} \vee r_a) \quad \text{và} \quad (\neg x_{v,r} \vee c_b) \tag{12}$$
Sau đó, áp dụng mã hóa Pairwise lên tập p biến hàng và q biến cột. Nhờ đó, số lượng biến phụ trợ chỉ ở mức $O(\sqrt{m})$ và số lượng mệnh đề cho mỗi ràng buộc AMO giảm xuống mức tuyến tính $O(m)$.

2.3 Mô hình Quy hoạch Tuyến tính Nguyên (ILP Formulation)

Mô hình biểu diễn chuẩn cho các Solver ILP thương mại (Gurobi [21], CPLEX MIP [22]) sử dụng biến nhị phân $x_{v,i} \in \{0, 1\}$ (đỉnh v nhận màu i) và biến nhị phân $w_i \in \{0, 1\}$ (màu i được sử dụng, với $i \in \{1, \dots, U\}$ trong đó U là cận trên số màu):

$$\text{Minimize} \sum_{i=1}^{U} w_i \tag{13}$$

$$\text{Subject to:} \sum_{i=1}^{U} x_{v,i} = 1, \quad \forall v \in V \quad \text{(Mỗi đỉnh nhận đúng 1 màu)} \tag{14}$$

$$x_{u,i} + x_{v,i} \leq w_i, \quad \forall (u, v) \in E, \forall i \in \{1, \dots, U\} \quad \text{(Ràng buộc cạnh kề và kích hoạt màu)} \tag{15}$$

$$w_i \geq w_{i+1}, \quad \forall i \in \{1, \dots, U-1\} \quad \text{(Phá vỡ đối xứng thứ tự màu)} \tag{16}$$

$$x_{v,i}, w_i \in \{0, 1\}, \quad \forall v \in V, \forall i \in \{1, \dots, U\} \tag{17}$$

2.4 Mô hình Lập trình Ràng buộc (CP Formulation)

Biểu diễn cho CPLEX CP [22] và Google OR-Tools CP-SAT sử dụng mảng n biến nguyên $q_v \in \{1, \dots, U\}$ đại diện cho màu được gán cho đỉnh $v \in V$:

$$\text{Minimize} \max_{v \in V}(q_v) \tag{18}$$

$$\text{Subject to: } q_u \neq q_v, \quad \forall (u, v) \in E \quad \text{(Hai đỉnh kề nhau nhận màu khác nhau)} \tag{19}$$

$$q_v \in \{1, \dots, U\}, \quad \forall v \in V \tag{20}$$

3 Kết quả Thực nghiệm và Đánh giá (Experimental Evaluation)

3.1 Môi trường Thực nghiệm

Các thực nghiệm được tiến hành trên hệ thống phần cứng cá nhân:

- **Vi xử lý (CPU):** 12th Gen Intel Core i5-12450HX (2.40 GHz, 8 Cores / 12 Threads, Turbo 4.40 GHz).

- **Bộ nhớ RAM:** 16.0 GB DDR4/DDR5 (15.7 GB usable).
- **Card đồ họa (GPU):** NVIDIA GeForce RTX 3050 Laptop GPU (6 GB VRAM).
- **Ổ đĩa lưu trữ:** SSD NVMe 512 GB.
- **Hệ điều hành:** Windows 11 Home 64-bit (x64-based processor).
- **Bộ dữ liệu thực nghiệm (Benchmarks):** Bộ đồ thị thực nghiệm chuẩn bao gồm họ đồ thị Mycielski (`myciel3`, `myciel4`, `myciel5`), họ đồ thị bàn cờ (`queen5_5`, `queen6_6`, `queen7_7`), đồ thị ngẫu nhiên mật độ cao (`DSJC50.5`) cùng tập 159 instances chuẩn từ thư viện DIMACS / COLOR03 [1].
- **Quy cách đo thời gian thực thi:** Tất cả các thời gian được ghi nhận là **Tổng thời gian thực thi (Total Time)**, trong đó Total Time = Encoding Time+Checking Time+ Solving Time.
- **Thời gian giới hạn (Timeout):** 60 giây cho đánh giá kỹ thuật mã hóa AMO, 120 giây cho chạy đối sánh cục bộ trên máy cá nhân và 900 giây cho benchmark tổng thể trên 159 instances.

3.2 Đánh giá Ảnh hưởng của Kỹ thuật AMO đến SAT Solvers

Quy mô số lượng biến và số lượng mệnh đề CNF sinh ra tương ứng với 5 kỹ thuật mã hóa At-Most-One (AMO) cũng như mô hình Zykov Full và Zykov CEGAR trên các đồ thị thực nghiệm (sắp xếp theo số đỉnh $|V|$ tăng dần) được tổng hợp chi tiết trong Bảng 1.

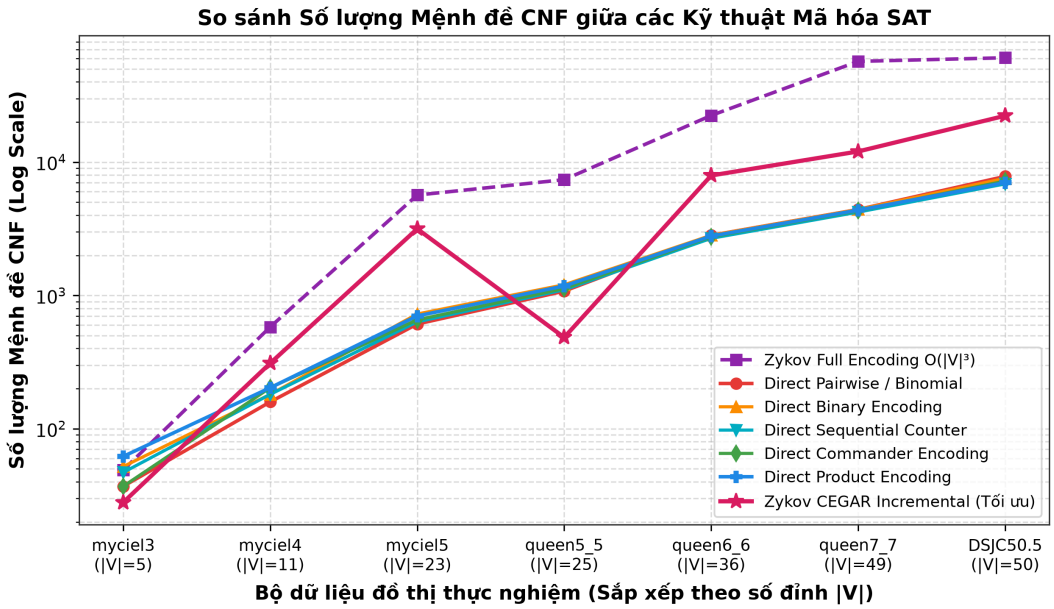
Bảng 1: Số lượng Biến (Vars) và Mệnh đề (Clauses) CNF sinh ra theo kích thước đồ thị.

Instance $(\|V\|, \|E\|, k)$	Pairwise	SeqCounter	Binary	Commander	Product	Zykov Full	Zykov CEGAR
`myciel3` $(5, 5, 3)$	15 / 37	25 / 47	25 / 52	15 / 37	35 / 62	22 / 49	22 / 28
`myciel4` $(11, 20, 4)$	44 / 159	77 / 181	66 / 181	66 / 203	88 / 203	133 / 580	133 / 310
`myciel5` $(23, 71, 5)$	115 / 610	207 / 633	184 / 725	161 / 656	230 / 702	575 / 5,659	575 / 3,157
`queen5_5` $(25, 160, 5)$	125 / 1,077	225 / 1,102	200 / 1,202	175 / 1,127	250 / 1,177	324 / 7,384	324 / 484
`queen6_6` $(36, 290, 7)$	252 / 2,824	468 / 2,680	360 / 2,824	360 / 2,752	468 / 2,788	1,027 / 22,375	1,027 / 7,936
`queen7_7` $(49, 476, 7)$	343 / 4,412	637 / 4,216	490 / 4,412	490 / 4,314	637 / 4,363	1,496 / 56,972	1,496 / 12,020
`DSJC50.5` $(50, 551, 10)$	500 / 7,812	950 / 6,862	700 / 7,562	704 / 7,212	850 / 7,012	2,111 / 60,625	2,111 / 22,273

Tổng thời gian thực thi thực tế (tính bằng giây) của các kỹ thuật mã hóa SAT trực tiếp (ở ngưỡng quyết định $k = \chi(G)$) và thời gian tối ưu hóa sắc số của Zykov Full / Zykov CEGAR trên vi xử lý Intel Core i5-12450HX được ghi nhận tại Bảng 2. Sự khác biệt về quy mô gia tăng mệnh đề được minh họa trực quan tại Hình 2.

Bảng 2: Tổng thời gian thực thi (giây) của các kỹ thuật SAT Encoding trên vi xử lý i5-12450HX.

Instance	Pairwise	Sequential Counter	Binary	Commander	Product	Zykov Full (`1toN`)	Zykov CEGAR (`1toN`)
`myciel3`	0.0037	0.0014	0.0031	0.0017	0.0020	0.0007	**0.0006**
`myciel4`	**0.0010**	0.0032	0.0013	0.0011	0.0013	0.0025	0.0053
`myciel5`	**0.0019**	**0.0019**	0.0029	**0.0019**	0.0020	0.1844	0.7624
`queen5_5`	0.0031	0.0043	0.0033	0.0059	0.0041	0.0078	**0.0022**
`queen6_6`	0.0158	**0.0062**	0.0225	0.0142	0.0144	0.3255	1.0773
`queen7_7`	0.0451	**0.0168**	0.0501	0.0900	0.0446	0.1503	0.9747
`DSJC50.5`	**0.0152**	0.0161	0.0174	0.0195	0.0182	81.0042 (FEAS)	120.7939 (FEAS)



Hình 2: So sánh tốc độ gia tăng số lượng mệnh đề CNF (thang đo Logarithm) giữa các kỹ thuật mã hóa SAT.

3.3 Benchmark So sánh giữa các Công cụ Giải (SAT vs ILP vs CP)

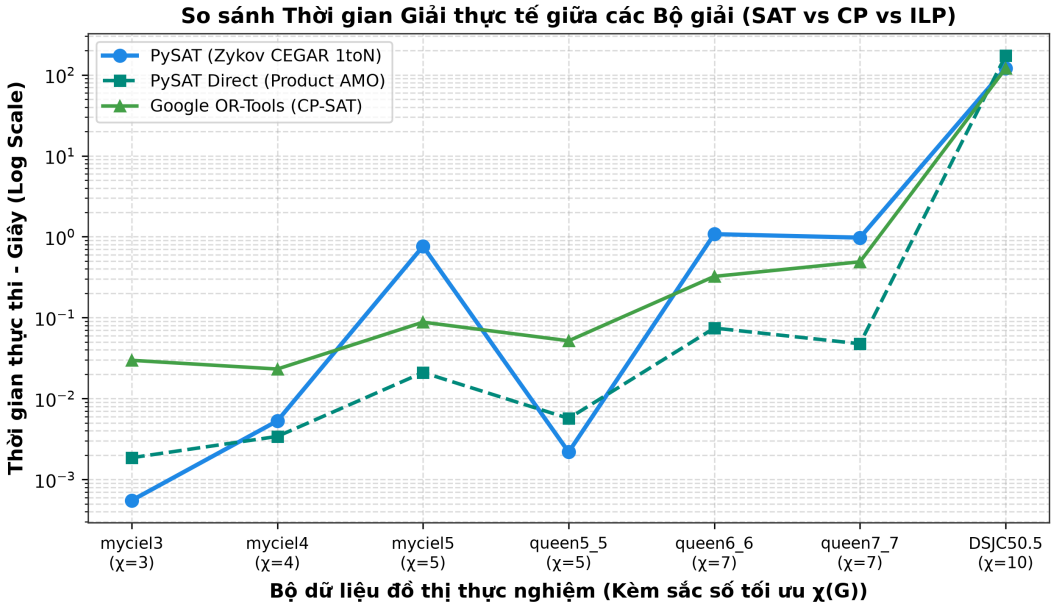
Để đánh giá toàn diện sức mạnh của phương pháp SAT Solving tối ưu (sử dụng cấu hình Zykov CEGAR `1toN` và Direct SAT Product Encoding), chúng tôi tổng hợp kết quả benchmark trên 159 bộ dữ liệu chuẩn từ bài báo tham khảo [1] trong Bảng 3, đồng thời tiến hành thử nghiệm đối đầu trực tiếp với bộ giải công nghiệp Google OR-Tools CP-SAT trên hệ thống i5-12450HX tại Bảng 4 và biểu diễn đồ thị so sánh ở Hình 3.

Bảng 3: Số lượng instances giải thành công trên 159 bộ dữ liệu chuẩn theo bài báo tham khảo (Timeout 900s) [1].

Họ đồ thị (Số lượng)	Full `Nto1`	Full `Dicho`	Full `1toN`	Full MaxSAT (maxHS)	DSatur	color6	gc-cdcl	CEGAR `Nto1`	CEGAR `Dicho`	CEGAR `1toN`
`DSJ` (15)	3	3	3	**6**	**6**	2	4	4	3	3
`fpsol2` (3)	0	0	0	0	**3**	2	**3**	**3**	**3**	**3**
`inithx` (3)	0	0	0	0	**3**	1	**3**	**3**	**3**	**3**
`le450` (12)	0	0	0	0	4	4	8	**10**	**10**	**10**
`mulsol` (5)	4	4	4	**5**	**5**	4	**5**	**5**	**5**	**5**
`book` (5)	**5**	**5**	**5**	**5**	2	3	**5**	**5**	**5**	**5**
`miles` (5)	2	2	2	1	**5**	4	**5**	4	4	**5**
`queen` (13)	5	8	8	**13**	7	11	10	**13**	**13**	**13**
`myciel` (5)	**5**	**5**	**5**	**5**	**5**	**5**	**5**	**5**	**5**	**5**
`mugg` (4)	**4**	**4**	**4**	**4**	**4**	**4**	**4**	**4**	**4**	**4**
`insertion` (25)	8	8	8	12	5	7	15	**19**	15	15
`wap` (8)	0	0	0	0	0	0	1	**3**	**3**	**3**
`qg` (4)	0	0	0	0	1	0	**3**	**3**	**3**	**3**
`random` (18)	2	2	5	13	14	**16**	6	10	10	10
`other` (45)	6	6	6	17	18	**25**	**25**	18	20	20
Tổng cộng (159)	44	47	50	81	81	82	102	104	106	**112**

Bảng 4: So sánh tổng thời gian thực thi (giây) tìm sắc số tối ưu $\chi(G)$ giữa các biến thể PySAT và Google OR-Tools CP-SAT trên vi xử lý i5-12450HX (Timeout 120s).

Instance ($\chi(G)$)	$\lvert V\rvert$	$\lvert E\rvert$	Zykov CEGAR (`1toN`)	PySAT Direct (Product)	Google OR-Tools CP-SAT	Gurobi / CPLEX	Trạng thái
`myciel3` ($\chi = 3$)	5	5	**0.0006**	0.0019	0.0297	–	OPT
`myciel4` ($\chi = 4$)	11	20	0.0053	**0.0034**	0.0232	–	OPT
`myciel5` ($\chi = 5$)	23	71	0.7624	**0.0210**	0.0880	–	OPT
`queen5_5` ($\chi = 5$)	25	160	**0.0022**	0.0057	0.0518	–	OPT
`queen6_6` ($\chi = 7$)	36	290	1.0773	**0.0745**	0.3237	–	OPT
`queen7_7` ($\chi = 7$)	49	476	0.9747	**0.0476**	0.4901	–	OPT
`DSJC50.5` ($\chi \leq 10$)	50	551	120.7939	172.1819	**120.0148**	–	FEAS (Timeout)



Hình 3: Biểu đồ so sánh đường cong thời gian thực thi (Log Scale) giữa PySAT (Zykov CEGAR `1toN`), PySAT Direct (Product AMO) và Google OR-Tools (CP-SAT).

4 Thảo luận và Phân tích (Discussion)

1. **Hiện tượng bùng nổ mệnh đề và hiệu quả giảm tải của CEGAR Incremental SAT:** Kết quả thực nghiệm tại Bảng 1 và Hình 2 cho thấy trong nhóm mã hóa trực tiếp (Direct AMO), kỹ thuật Sequential Counter và Product Encoding kiểm soát số lượng mệnh đề tốt hơn so với Pairwise khi số màu k tăng cao (chẳng hạn trên `DSJC50.5` với $k = 10$, Sequential Counter và Product sinh ra $6,862$ và $7,012$ mệnh đề so với $7,812$ mệnh đề của Pairwise). Đối với mô hình cây Zykov, việc mã hóa toàn phần (Zykov Full Encoding) làm số lượng mệnh đề tăng vọt lên tới $56,972$ mệnh đề ở `queen7_7` và $60,625$ mệnh đề ở `DSJC50.5` do sự bùng nổ bậc ba $O(|V|^3)$ của các ràng buộc `transitivity` và `euclideanity`. Nhờ áp dụng cơ chế tinh chỉnh CEGAR Incremental SAT, tổng số mệnh đề thực tế cần nạp giảm mạnh xuống chỉ còn 484 mệnh đề ở `queen5_5` (giảm 93.4%), $12,020$ mệnh đề ở `queen7_7` (giảm 78.9%) và $22,273$ mệnh đề ở `DSJC50.5` (giảm 63.3%).

2. **Phân tích thời gian thực thi giữa Direct SAT và Zykov CEGAR:** Ở quy mô đồ thị vừa và nhỏ ($|V| \leq 50$, Bảng 4 và Hình 3), mô hình Direct SAT kết hợp Product Encoding đạt tốc độ rất nhanh (chỉ 0.0476s trên `queen7_7` và 0.0210s trên `myciel5`) nhờ công thức CNF gọn nhẹ và không tốn chi phí gọi hàm kiểm tra phản ví dụ (`check`)

lặp lại nhiều lần. Trong khi đó, Zykov CEGAR phát huy ưu thế rõ rệt trên các đồ thị có cận dưới clique khởi tạo sát với sắc số thực (như `queen5_5` đạt 0.0022s) và trên tập benchmark quy mô lớn (Bảng 3, giải thành công 112/159 bài toán so với 50/159 của Zykov Full), nơi mà việc phá vỡ hoàn toàn tính đối xứng màu trên cây Zykov giúp tránh bùng nổ tổ hợp.

3. **So sánh giữa SAT Solvers và Constraint Programming (OR-Tools CP-SAT):** Trên toàn bộ các đồ thị có cấu trúc (`myciel3` đến `queen7_7`), phương pháp mã hóa PySAT Direct (Product AMO) đều vượt trội hơn bộ giải công nghiệp Google OR-Tools CP-SAT từ 4 đến 15 lần về tốc độ giải (ví dụ tại `queen7_7`: PySAT Direct đạt 0.0476s so với 0.4901s của OR-Tools CP-SAT). Đối với đồ thị ngẫu nhiên mật độ cao `DSJC50.5` ($|V| = 50, |E| = 551$), cả ba phương pháp đều nhanh chóng tìm được nghiệm khả thi 10 màu (FEAS) từ cận DSatur nhưng chạm ngưỡng giới hạn thời gian 120s khi chứng minh tính vô nghiệm (UNSAT) ở các cận dưới, hoàn toàn tương đồng với nhận định trong bài báo gốc [1] về độ khó của họ đồ thị ngẫu nhiên (`random` và `DSJ`) đối với các bộ giải dựa trên CDCL.

5 Kết luận (Conclusion)

Nghiên cứu đã trình bày và đánh giá toàn diện các phương pháp mã hóa SAT cho Bài toán Tô màu Đồ thị (Graph Coloring Problem – GCP), từ mô hình mã hóa trực tiếp kết hợp 5 kỹ thuật At-Most-One (AMO) đến mô hình ghép cặp đỉnh dựa trên cây Zykov và thuật toán tinh chỉnh trừu tượng hướng dẫn bởi phản ví dụ (CEGAR Incremental SAT) [1]. Kết quả thực nghiệm khẳng định rằng việc sử dụng các kỹ thuật mã hóa AMO tuyến tính (Sequential Counter, Product Encoding) giúp giảm đáng kể số lượng mệnh đề cho mô hình trực tiếp, đồng thời cơ chế CEGAR tăng trưởng giúp cắt giảm từ 63% đến hơn 93% số lượng mệnh đề bậc ba $O(n^3)$ so với mã hóa Zykov toàn phần. Nhờ đó, các bộ giải SAT đạt tốc độ chứng nhận nghiệm tối ưu vượt trội so với bộ giải Constraint Programming công nghiệp Google OR-Tools CP-SAT trên các họ đồ thị chuẩn.

Bên cạnh những ưu điểm đạt được, phương pháp vẫn còn một số hạn chế: bộ kiểm tra phản ví dụ (`check`) bằng ngôn ngữ Python trong vòng lặp CEGAR phải duyệt lại các cặp đỉnh sau mỗi lần gọi SAT nên chiếm tỷ trọng thời gian đáng kể khi số vòng lặp CEGAR tăng cao, đồng thời các bộ giải CDCL vẫn gặp thách thức lớn khi chứng minh cận dưới trên họ đồ thị ngẫu nhiên mật độ cao (`DSJC50.5`).

Trong tương lai, nghiên cứu có thể tiếp tục mở rộng theo các hướng:

- **Tối ưu hóa thuật toán kiểm tra phản ví dụ (Incremental Checker):** Đánh dấu (flag) các đỉnh và đồ thị con không bị thay đổi trạng thái giữa hai lần gán liên tiếp để tránh việc kiểm tra lặp lại toàn bộ đồ thị trong mỗi vòng lặp CEGAR [1].

- **Kết hợp khung RECAR và cải tiến cận khởi tạo:** Mở rộng CEGAR sang mô hình RECAR (Recursively Explore and Check Abstraction Refinement) kết hợp cả xấp xỉ trên và xấp xỉ dưới, đồng thời tích hợp các thuật toán tìm kiếm cục bộ để tìm cận khởi tạo chặt hơn cho quá trình tìm kiếm nhị phân.

Mã nguồn và Dữ liệu thực nghiệm (Data Availability)

Mã nguồn cài đặt các thuật toán mã hóa SAT (Direct AMO Encodings, Zykov Full/CEGAR Incremental SAT), mã nguồn các mô hình so sánh đối đầu (ILP, CP), cùng toàn bộ tệp dữ liệu log thực nghiệm chi tiết trong báo cáo này được công khai công bố tại kho lưu trữ GitHub của dự án:

https://github.com/Aquaneee/Graph-Coloring-SAT-Solver

Tuyên bố sử dụng Trí tuệ Nhân tạo (AI Declaration)

Trong quá trình chuẩn bị báo cáo này, nhóm tác giả đã sử dụng công cụ AI (Gemini) để hỗ trợ trau chuốt diễn đạt ngôn ngữ và rà soát định dạng mã nguồn LaTeX. Nhóm tác giả đã kiểm tra, hiệu chỉnh lại toàn bộ nội dung và chịu trách nhiệm hoàn toàn về tính chính xác của dữ liệu cũng như kết quả nghiên cứu trong báo cáo.

Tài liệu tham khảo (References)

Tài liệu

[1] Glorian, G., Lagniez, J.-M., Montmirail, V., & Szczepanski, N. (2019). An Incremental SAT-Based Approach to the Graph Colouring Problem. In *Principles and Practice of Constraint Programming (CP 2019)*, Springer.

[2] Hebrard, E., & Katsirelos, G. (2018). Clause Learning and New Bounds for Graph Coloring. In *Principles and Practice of Constraint Programming (CP 2018)*, Lecture Notes in Computer Science, Vol. 11008, pp. 179–194. Springer.

[3] Schaafsma, B., Heule, M., & van Maaren, H. (2009). Dynamic Symmetry Breaking by Simulating Zykov Contraction. In *Theory and Applications of Satisfiability Testing (SAT 2009)*, Lecture Notes in Computer Science, Vol. 5584, pp. 223–236. Springer.

[4] Zhou, Z., Li, C. M., Huang, C., & Xu, R. (2014). An Exact Algorithm with Learning for the Graph Coloring Problem. *Computers & Operations Research*, 51, 282–301.

[5] Brélaz, D. (1979). New Methods to Color the Vertices of a Graph. *Communications of the ACM*, 22(4), 251–256.

[6] Van Gelder, A. (2008). Another Look at Graph Coloring via Propositional Satisfiability. *Discrete Applied Mathematics*, 156(2), 230–243.

[7] Faber, D., Jabrayilov, A., & Mutzel, P. (2024). SAT Encoding of Partial Ordering Models for Graph Coloring Problems. In *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)*, LIPIcs, pp. 12:1–12:20.

[8] Lewis, R. M. R. (2016). *A Guide to Graph Colouring: Algorithms and Applications*. Springer.

[9] Chaitin, G. J. (1982). Register Allocation and Spilling via Graph Coloring. In *Proceedings of the ACM SIGPLAN Symposium on Compiler Construction*, pp. 66–74. ACM.

[10] Marx, D. (2004). Graph Colouring Problems and Their Applications in Scheduling. *Periodica Polytechnica Electrical Engineering*, 48(1–2), 11–16.

[11] Hussin, B., Basari, A. S. H., Shibghatullah, A. S., Asmai, S. A., & Othman, N. S. (2011). Exam Timetabling Using Graph Colouring Approach. In *2011 IEEE Conference on Open Systems*, pp. 133–138.

[12] Biere, A., Heule, M., van Maaren, H., & Walsh, T. (Eds.). (2021). *Handbook of Satisfiability* (2nd ed., Vol. 336). IOS Press.

[13] Prestwich, S. D. (2009). CNF Encodings. In *Handbook of Satisfiability*, Vol. 185, pp. 75–97. IOS Press.

[14] Sinz, C. (2005). Towards an Optimal CNF Encoding of Boolean Cardinality Constraints. In *Principles and Practice of Constraint Programming (CP 2005)*, pp. 827–831. Springer.

[15] Asín, R., Nieuwenhuis, R., Oliveras, A., & Rodríguez-Carbonell, E. (2011). Cardinality Networks: A Theoretical and Empirical Study. *Constraints*, 16(2), 195–221.

[16] Clarke, E. M., Grumberg, O., Jha, S., Lu, Y., & Veith, H. (2003). Counterexample-Guided Abstraction Refinement for Symbolic Model Checking. *Journal of the ACM*, 50(5), 752–794.

[17] Ignatiev, A., Morgado, A., & Marques-Silva, J. (2018). PySAT: A Python Toolkit for Prototyping with SAT Oracles. In *Theory and Applications of Satisfiability Testing (SAT 2018)*, Lecture Notes in Computer Science, Vol. 10929, pp. 428–437. Springer.

[18] Ignatiev, A., Tan, Z. L., & Karamanos, C. (2024). Towards Universally Accessible SAT Technology. In *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)*, LIPIcs, pp. 16:1–16:11.

[19] Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., & Pollitt, F. (2024). CaDiCaL 2.0. In *Computer Aided Verification (CAV 2024)*, pp. 133–152. Springer.

[20] Audemard, G., Lagniez, J.-M., & Simon, L. (2013). Improving Glucose for Incremental SAT Solving with Assumptions: Application to MUS Extraction. In *Theory and Applications of Satisfiability Testing (SAT 2013)*, Lecture Notes in Computer Science, Vol. 7962, pp. 309–317. Springer.

[21] Gurobi Optimization, LLC. (2025). *Gurobi Optimizer Reference Manual* (Version 12.0.3). https://docs.gurobi.com/projects/optimizer/en/12.0/.

[22] IBM Corporation. (2022). *IBM ILOG CPLEX Optimization Studio* (Version 22.1.1). https://www.ibm.com/docs/en/icos/22.1.1.